

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: K. Sirisha

Roll No.: A24126552260

Section: B

1. TITLE

To-Do List Manager Using DOM Manipulation and Event Listeners

2. AIM

To develop a To-Do List web application that allows a user to add tasks, mark them as complete, and delete them dynamically using JavaScript and DOM manipulation.

3. OBJECTIVE

- To capture user input from a text field and an 'Add Task' button.
- To dynamically create and insert list item elements into the DOM.
- To attach event listeners for click and keyboard (Enter key) events.
- To toggle a task's completed state and update its label and style accordingly.
- To remove a task element from the DOM when the Delete button is clicked.
- To show or hide an 'empty list' message based on the number of tasks present.

4. THEORY

The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can read and modify at runtime. Methods such as `document.createElement()`, `element.appendChild()` and `element.remove()` allow new elements to be built and inserted into, or removed from, the page without reloading it.

Event listeners, attached using `element.addEventListener(event, handler)`, let the page respond to user actions such as a button click or a key press. In this application, clicking the 'Add Task' button (or pressing Enter inside the input field) reads the entered text, validates that it is not empty, and creates a new list item (`<li>`) containing the task text and a group of two buttons: 'Complete' and 'Delete'.

The `classList.toggle()` method is used to add or remove a CSS class that applies a strikethrough style to a task, switching it between its pending and completed appearance. When the task list becomes empty (either initially or after deleting all tasks), a message is displayed to indicate that no tasks are available.

5. ALGORITHM / PROCEDURE

1. Create a new HTML file with a text input, an 'Add Task' button, an empty unordered list, and an 'empty list' message paragraph.
2. Select the input field, add-task button, task list and empty-message elements using `document.getElementById()`.
3. Write an `updateEmptyMessage()` function that shows the empty message when the list has no children, and hides it otherwise.

4. Attach a click event listener to the 'Add Task' button.
5. Inside the listener, read and trim the input value; if it is empty, alert the user and stop.
6. Create a new element with a span for the task text and a button group containing 'Complete' and 'Delete' buttons.
7. Attach a click event listener to the Complete button that toggles a 'completed' CSS class on the task text and updates the button label.
8. Attach a click event listener to the Delete button that removes the task item from the DOM and calls updateEmptyMessage().
9. Append the task item to the task list, clear the input field, and call updateEmptyMessage().
10. Attach a keydown event listener to the input field so pressing Enter triggers the same add-task action.
11. Save the file, open it in a browser, and add, complete and delete tasks to verify the behaviour.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My To-Do List</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>My To-Do List</h1>
    <div class="input-section">
      <input type="text" id="taskInput" placeholder="Enter a task.....">
      <button id="addTaskBtn">Add Task</button>
    </div>
    <ul id="taskList"></ul>
    <p id="emptyMessage">No tasks available.</p>
  </div>
  <script src="Task3.js"></script>
  <style>
    * { box-sizing: border-box; }
    body { margin: 0; font-family: Arial, sans-serif; background: #f2f4f7; }
    .container {
      width: 600px; margin: 60px auto; padding: 30px;
      background: white; border-radius: 10px;
      box-shadow: 0 4px 15px rgba(0, 0, 0, 0.15);
    }
    h1 { text-align: center; margin-bottom: 25px; }
    .input-section { display: flex; gap: 10px; }
    #taskInput {
      flex: 1; padding: 12px; border: 1px solid #aaa;
      border-radius: 5px; font-size: 16px;
    }
    #addTaskBtn {
      padding: 12px 18px; border: none; border-radius: 5px;
      background: #007bff; color: white; cursor: pointer; font-size: 16px;
    }
    #addTaskBtn:hover { background: #0056b3; }
    #taskList { list-style: none; padding: 0; margin-top: 25px; }
    .task-item {
      display: flex; align-items: center; justify-content: space-between;
      padding: 12px; margin-bottom: 10px; border: 1px solid #ddd;
      border-radius: 5px; background: #f9f9f9;
    }
```

```

}
.task-text { flex: 1; font-size: 17px; }
.completed { text-decoration: line-through; color: gray; }
.button-group { display: flex; gap: 5px; }
.complete-btn, .delete-btn {
  padding: 8px 12px; border: none; border-radius: 4px;
  color: white; cursor: pointer;
}
.complete-btn { background: #28a745; }
.complete-btn:hover { background: #1e7e34; }
.delete-btn { background: #dc3545; }
.delete-btn:hover { background: #a71d2a; }
#emptyMessage { text-align: center; color: gray; margin-top: 20px; }
</style>
</body>
</html>

```

Task3.js

```

// Select HTML elements using DOM
const taskInput = document.getElementById("taskInput");
const addTaskBtn = document.getElementById("addTaskBtn");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

// Function to update empty message
function updateEmptyMessage() {
  if (taskList.children.length === 0) {
    emptyMessage.style.display = "block";
  } else {
    emptyMessage.style.display = "none";
  }
}

// Add task button event listener
addTaskBtn.addEventListener("click", function () {
  const taskText = taskInput.value.trim();

  // Check if task is empty
  if (taskText === "") {
    alert("Please enter a task.");
    return;
  }

  // Create new list item
  const taskItem = document.createElement("li");
  taskItem.className = "task-item";

  // Create task text
  const task = document.createElement("span");
  task.className = "task-text";
  task.textContent = taskText;

  // Create button group
  const buttonGroup = document.createElement("div");
  buttonGroup.className = "button-group";

  // Create Complete button
  const completeBtn = document.createElement("button");
  completeBtn.textContent = "Complete";
  completeBtn.className = "complete-btn";

  // Complete button event
  completeBtn.addEventListener("click", function () {

```

```

        task.classList.toggle("completed");
        if (task.classList.contains("completed")) {
            completeBtn.textContent = "Completed";
        } else {
            completeBtn.textContent = "Complete";
        }
    });

    // Create Delete button
    const deleteBtn = document.createElement("button");
    deleteBtn.textContent = "Delete";
    deleteBtn.className = "delete-btn";

    // Delete button event
    deleteBtn.addEventListener("click", function () {
        taskItem.remove();
        // Show empty message if there are no tasks
        updateEmptyMessage();
    });

    // Add buttons to button group
    buttonGroup.appendChild(completeBtn);
    buttonGroup.appendChild(deleteBtn);

    // Add task and buttons to list item
    taskItem.appendChild(task);
    taskItem.appendChild(buttonGroup);

    // Add list item to task list
    taskList.appendChild(taskItem);

    // Clear input box
    taskInput.value = "";

    // Update empty message
    updateEmptyMessage();
});

// Allow Enter key to add task
taskInput.addEventListener("keydown", function (event) {
    if (event.key === "Enter") {
        addTaskBtn.click();
    }
});

// Display empty message initially
updateEmptyMessage();

```

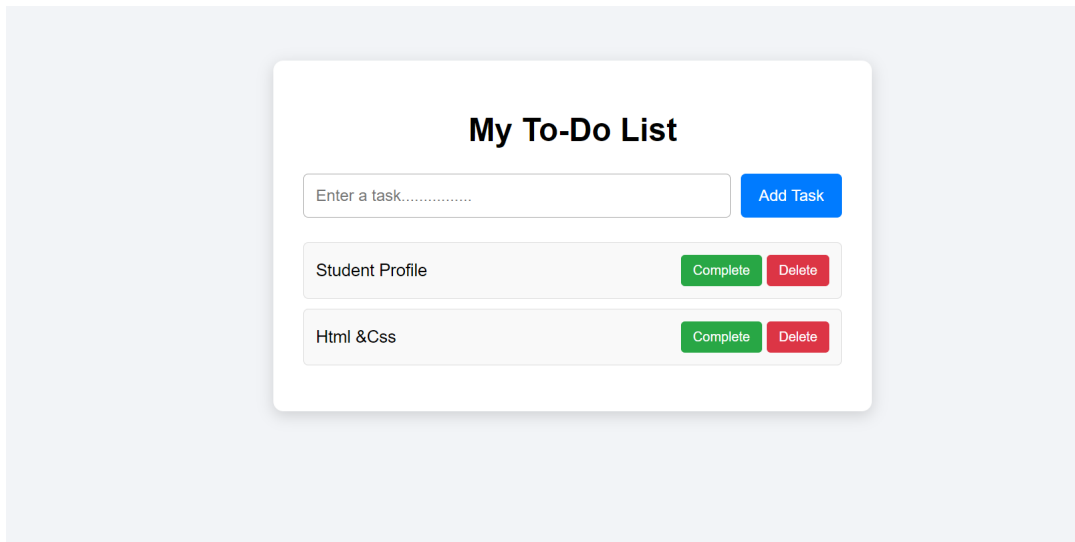
7. CODE EXPLANATION

Code	Explanation
<code>document.getElementById(...)</code>	Selects the input field, Add Task button, task list and empty-message elements from the DOM.
<code>addTaskBtn.addEventListener("click", ...)</code>	Listens for a click on the 'Add Task' button to trigger task creation.
<code>.value.trim()</code>	Reads the current text entered in the input field and removes extra whitespace.
<code>if (taskText === "") { alert(...); return; }</code>	Validates that the input is not empty before creating a task.
<code>document.createElement("li") / ("span") / ("div") / ("button")</code>	Creates the new task item, its text span, the button group, and the Complete/Delete buttons.
<code>completeBtn.addEventListener("click", ...)</code>	Toggles the 'completed' class on the task text and updates the button label between Complete and Completed.
<code>classList.toggle("completed")</code>	Adds or removes the CSS class that applies the strikethrough style to a finished task.
<code>deleteBtn.addEventListener("click", ...)</code>	Removes the task item from the DOM using <code>taskItem.remove()</code> and refreshes the empty message.
<code>taskList.appendChild(taskItem)</code>	Attaches the newly created task item to the visible task list.
<code>updateEmptyMessage()</code>	Shows the 'No tasks available.' message when the list is empty, and hides it otherwise.
<code>taskInput.addEventListener("keydown", ...)</code>	Allows a task to be added by pressing the Enter key inside the input field.

8. TEST CASES AND EXECUTION DATA

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Load the page in browser	Empty task list is shown with the 'No tasks available.' message	Displayed correctly	Pass
TC-02	Click 'Add Task' with the input left empty	An alert asks the user to enter a task	Alert displayed as expected	Pass
TC-03	Type 'Student Profile' and click 'Add Task'	A new task item appears with Complete and Delete buttons, input clears	Task added and input cleared	Pass
TC-04	Type 'Html &Css' and press Enter	A new task is added without clicking the button	Task added correctly via Enter key	Pass
TC-05	Click 'Complete' on a task	Task text gets a strikethrough style and the button label changes to 'Completed'	Style and label updated correctly	Pass
TC-06	Click 'Delete' on a task	That task is removed from the list	Task removed successfully	Pass
TC-07	Delete all tasks from the list	The 'No tasks available.' message reappears	Empty message displayed again	Pass

9. OUTPUT



10. RESULT

Thus, a To-Do List web application was successfully developed using DOM manipulation and event listeners, allowing tasks to be added, marked complete and deleted dynamically. The output was verified against the expected results for all test cases.